



COMILLAS

UNIVERSIDAD PONTIFICIA

ICAI

ICADE

CIHS

GUIA PRACTICA 1 – Rest API

Desarrollo de aplicaciones y servicios

Introducción

En el desarrollo de esta práctica, cada grupo va a realizar la implementación de la REST API para el proyecto final de la asignatura.

El proyecto que realizaremos a lo largo de la asignatura será una copia de ChatGPT. Se montará un modelo de negocio en torno a un LLM.

La idea es que en nuestra aplicación un usuario pueda registrarse, pueda iniciar un chat con el LLM, recuperar chats antiguos y borrarlos.

Así mismo, el usuario tiene un límite de mensajes que puede enviar en total al mes, siendo este límite autorenovable cada mes.

La aplicación frontend contará con una parte de ajustes de usuario en la cual el usuario podrá cambiar su contraseña, datos personales y demás.

El grupo tiene libertad a la hora de decidir si quiere especializar su LLM en torno a un prompt específico.

En prácticas posteriores, montaremos la aplicación frontend que consumirá lo desarrollado en esta práctica.

Repositorio

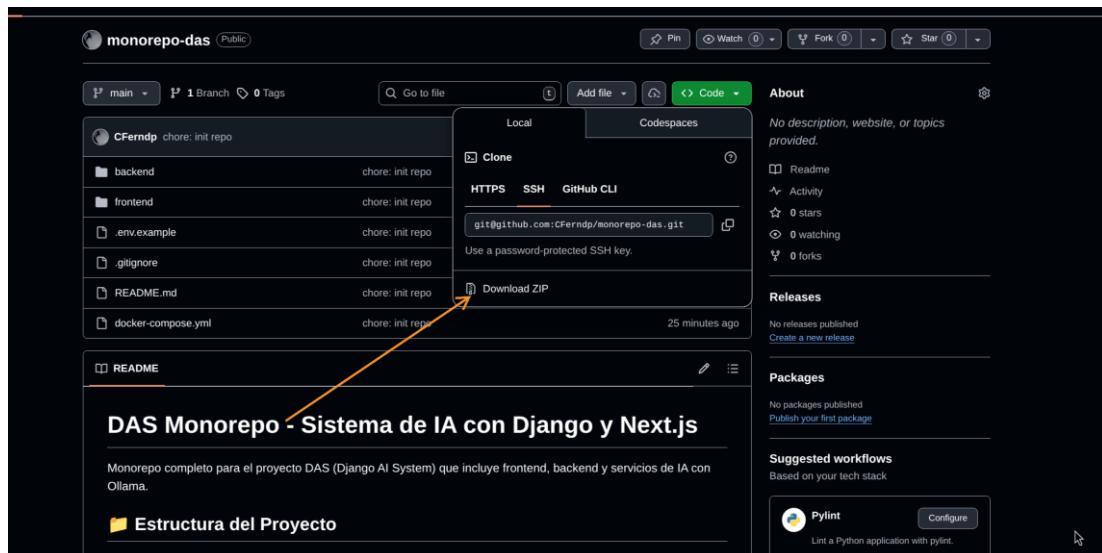
Se ha preparado un repositorio base del cual el grupo deberá descargarse. Dicho repositorio sigue el patrón de *monorepo*, en el cual el proyecto de **frontend** y **backend** viven. Así podemos configurar un docker que ejecute y comunique ambas partes del proyecto. Este repositorio será el que se use a lo largo de las prácticas.

Incluye un fichero docker compose que ejecuta:

- Ollama para servir un LLM
- Backend para ejecutar la lógica de negocio
 - Incluye configuraciones de base de datos, CORS y documentación con Swagger.
- Frontend para el usuario final. Aplicación NextJS limpia.

Acceda a la siguiente URL: <https://github.com/CFerndp/monorepo-das>

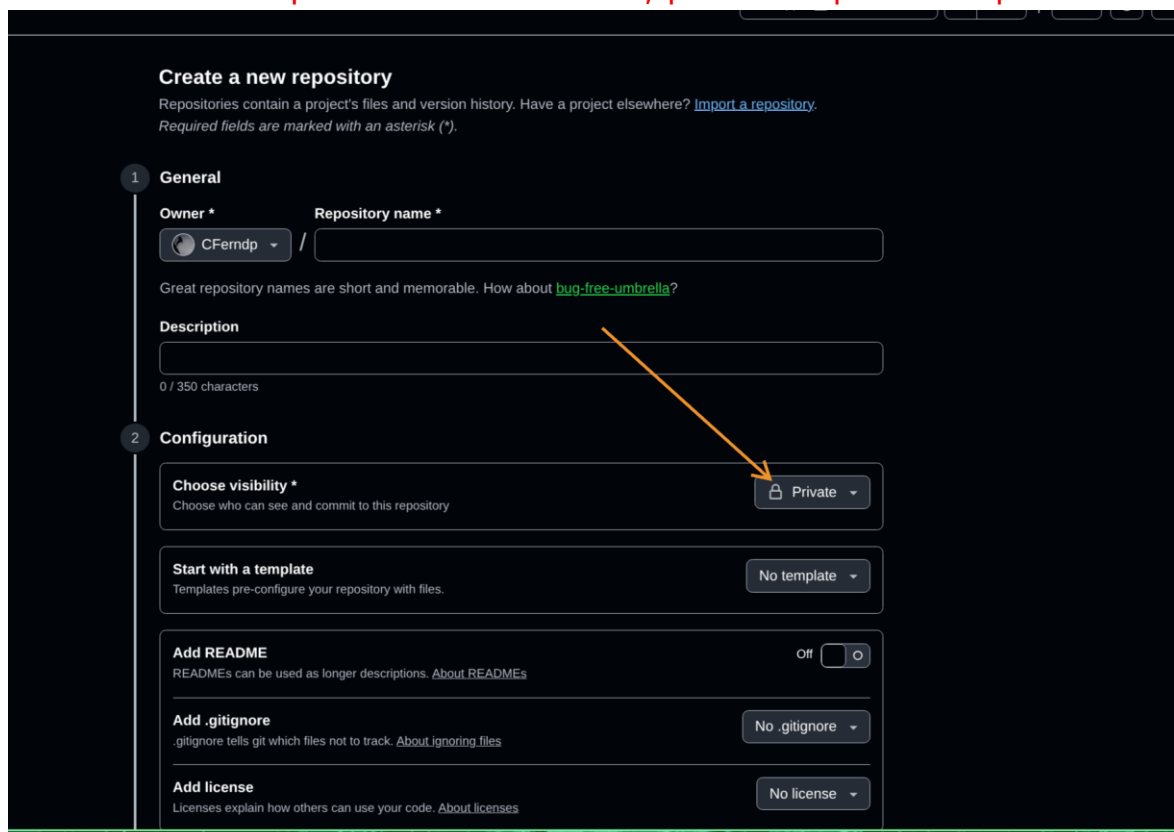
Pinche en el botón "<> Code" y seleccione la opción "Download ZIP"



A continuación, cree un repositorio en Github e incluya el contenido descargado. Para ello:

1. Descomprima el ZIP descargado
2. Borre la carpeta .git que hay dentro. Este paso es importante para que no se vincule el git al repositorio que os proporcionamos de plantilla.

Cuando configure el nuevo repositorio de código, debe configurar que la visibilidad sea privada. De no ser así, podría suspender la práctica.



Una vez creado el repositorio, añada como colaboradores a los profesores de la asignatura:

- Álvaro Ruiz Calzada (si pertenece al grupo A): **arcalzada**
- Cristian Fernández del Pozo: (si pertenece al grupo B): **cferndp**
- Mario Vidal Dominguez (obligatorio): **mariovido**

La no inclusión de los profesores impedirá el acceso al repositorio y a la corrección.

Podríamos conseguir lo mismo haciendo un *fork* del proyecto, pero Github no permite cambiar la visibilidad de un *fork*, copia la del repositorio de origen.

Por último, el repositorio cuenta con un fichero *Readme* que incluye instrucciones básicas para el uso del fichero *docker-compose* que hay. Gracias a este fichero, podemos ejecutar un servidor con *ollama* y ejecutar un LLM. Así mismo, dentro de la carpeta de *backend*, se encuentra una función llamada *chat_llm* que es la encargada de realizar la conexión con el LLM.

Revise dicho *readme* y el fichero *docker compose* para su uso.

Organización del trabajo

Se ha de crear en Git Project un nuevo *milestone* con nombre “Sprint 1” donde se realizará la distribución de tareas entre los integrantes del grupo de trabajo.

Se creará la documentación del sprint donde se debe de indicar de manera clara y resumida:

- Qué tareas se van a realizar.
- Organización de la carga.
- Dificultades encontradas en su realización.
- Desviaciones que se hayan podido producir.

El desarrollo de esta práctica se realizará en la rama “feature/p1”, siguiendo la metodología de trabajo de gitFlow.

Se evaluará únicamente lo que esté presente en la rama *main* en el momento de la entrega.

Base de datos

Para la implementación de nuestra Rest API con Django Rest Framework, deberemos crear la base de datos acorde a la siguiente imagen.

Recordad que tanto la gestión como la implementación de las tablas en la base de datos se realiza mediante el *ORM* de Django, creando modelos, migraciones y ejecutando dichas migraciones.

Tablas de la base de datos:

Usage

- **id:** id de la entrada. Automático.
- **messages_used:** Número de mensajes usados.
- **messages_limit:** Número de límite de mensajes.
- **reset_date:** Fecha de reset del límite.
- **user_id:** usuario al que refiere.

Chats

- **id:** id de la entrada. Automático.
- **title:** Título del chat.
- **created_at:** Fecha de creación del chat.
- **user_id:** Usuario que inició el chat.

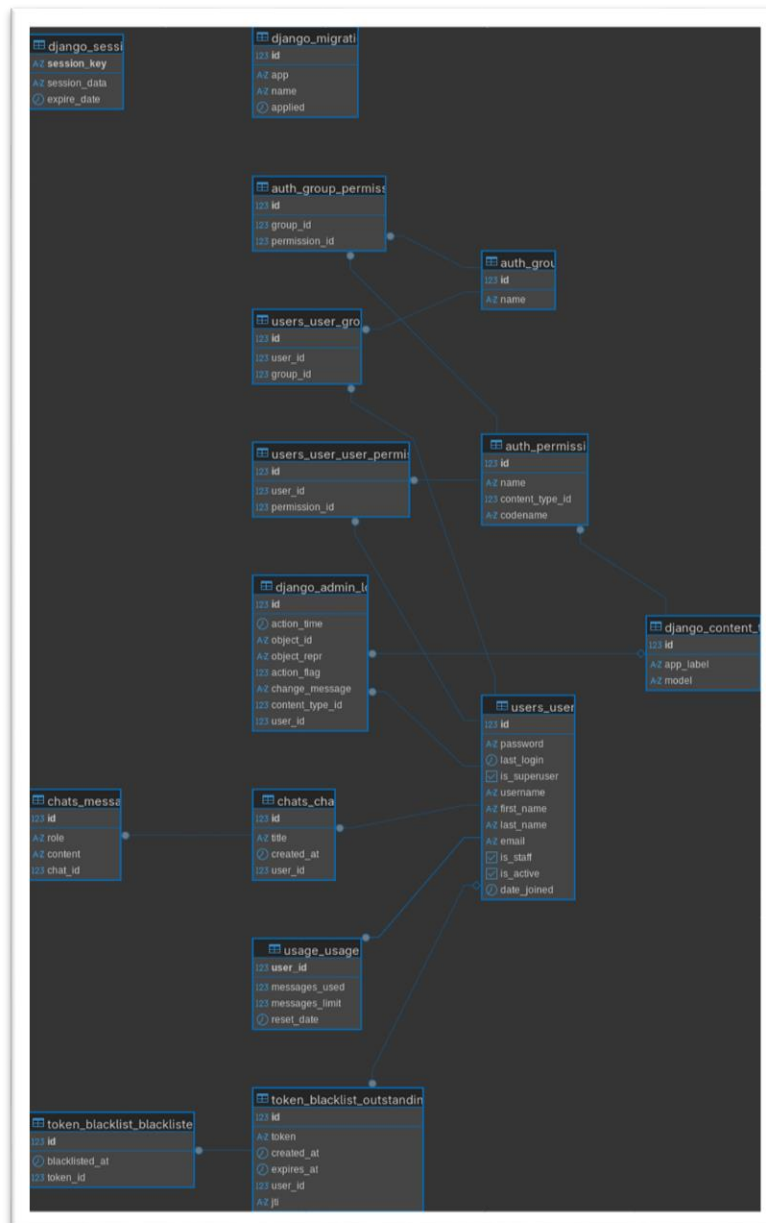
Chat Message

- **Id:** id de la entrada. Automático.
- **role:** system or user.
- **content:** contenido del mensaje.
- **chat_id:** id del chat al que pertenece.

Las relaciones son las siguientes:

- **1 User → 1 Usage (1-1):** Un usuario tiene un único registro de uso.
- **1 User → N Chats (1-N):** Un usuario puede tener múltiples chats.
- **1 Chat → N Chat Messages (1-N):** Un chat contiene múltiples mensajes.

El resto de tablas se generan de manera automática por la gestión de usuario y por el uso de JWT.



Endpoints a implementar

Para dar soporte a la futura aplicación frontend, se deben implementar una serie de endpoints que permitan el hacer CRUD de los datos, siguiendo una lógica de negocio. Se sugieren los siguientes endpoints:

Auth (autogenerados)

auth		^
POST	/api/auth/login/	✓
POST	/api/auth/refresh/	✓

Chat

chats		^
GET	/api/chats/	🔒 ✓
POST	/api/chats/	🔒 ✓
GET	/api/chats/{chat_id}/	🔒 ✓
POST	/api/chats/{chat_id}/	🔒 ✓
DELETE	/api/chats/{chat_id}/	🔒 ✓

Usage

usage		^
GET	/api/usage/	🔒 ✓

Users

users		^
POST	/api/users/logout	🔒 ✓
GET	/api/users/profile	🔒 ✓
PUT	/api/users/profile	🔒 ✓
PUT	/api/users/profile/password	🔒 ✓
POST	/api/users/register/	🔒 ✓

Validaciones y reglas de negocio

A continuación definiremos las reglas de negocio que se tienen que cumplir, así como las validaciones pertinentes.

Chat

Un usuario no podrá escribir más mensajes en un chat si ya ha superado el límite de mensajes permitidos en el mes.

Si el límite está superado, pero la fecha de hoy es mayor a la fecha de renovación, el mensaje se podrá ejecutar y enviar y la fecha se actualizará a dentro de un mes. Así mismo, el número de mensajes usados se reiniciará.

Usuario

A la hora de registrarse, se aportarán dos contraseñas y se comprobará que dichas contraseñas cumplan las siguientes condiciones:

- Más de 8 caracteres
- Incluye, al menos, una mayúscula.
- Incluye, al menos, una minúscula.

Así mismo, se comprobará que el username no existe previamente y que el email introducido tiene un formato correcto (contiene texto, @, texto y .com|.es, etc.)

Entregable

Para la entrega de esta práctica se creará una actividad en el aula virtual. En la tarea adjuntad una memoria con la documentación pertinente, así como con el enlace al repositorio. Recordad que sólo se corregirá lo que hay en la rama *main*. No se evaluará código entregado por otros medios.

Cada día de retraso en la entrega resta 0.5 puntos de la nota final. A partir de 3 días, la práctica no se corregirá y estará suspensa (0)

Anexo I: Criterios de evaluación

La evaluación de la práctica se realizará conforma a los criterios expuestos a continuación:

- Creación de hitos.
- Documentación y reparto de tareas.
- Modelado correcto según lo descrito (ORM)
- Funcionalidades de negocio descritas implementadas.
- Validaciones correctas.
- Respuesta de los endpoints correcta.
- Ejecución de docker-compose correcta.
 - Para corregir, los profesores ejecutaremos el comando `docker compose up` que levantará tanto el *backend*, como *ollama*. Comprobad previamente que funciona correctamente.

Nota final

El uso de la IA para la realización de la asignatura está permitido, pero bajo criterios de uso justo. Si se ve un uso abusivo de la misma, se podría suspender la práctica (documentación sin sentido, código no consistente, etc.)